A step-by-step guide to reinforcement learning evidence accumulation models of instrumental learning tasks

Steven Miletić, Niek Stevenson, (some other people probably), Andrew Heathcote

25 July, 2026

1 Introduction

In this tutorial, we demonstrate how to design combinations of reinforcement learning and evidence accumulation models (RL-EAMs) and fit them to data from instrumental learning tasks using hierarchical Bayesian methods, using the EMC2 package (Stevenson et al. 2026). We assume familiarity with the basic EMC2 workflow — readers new to the package are encouraged to first consult the companion tutorial on dynamic EAMs (Miletić et al. 2026), which covers model specification, prior specification, sampling, convergence checking, and posterior predictive evaluation in detail. The present tutorial builds directly on those foundations, focusing on the additional steps required for RL-EAMs: covariate specification, kernel definition, accumulator coding, and feedback generation.

To enhance readability, the code blocks below include only the arguments strictly required to run the code. The R Markdown file that generates this document additionally contains hidden code blocks handling multi-core processing, automatic saving and loading of samples, and plot formatting. The source document is available at https://github.com/StevenM1/rl_eam_tutorial/, alongside an HTML version that facilitates copy-and-pasting.

First, let's load the required packages and data:

```
# TMP -- install the right branch
# remotes::install_github("ampl-psych/EMC2@trend_api",dependencies=TRUE,
Ncpus=8);.rs.restartR()
# END TMP
library(EMC2)
set.seed(1) # for reproducibility

# Plotting functions used later in this tutorial
code_url <- paste0(
  "https://raw.githubusercontent.com/StevenM1/",
  "rl_eam_tutorial/refs/heads/main/RL_plotting_utils.R")
source(code_url)

# Load dataset 1 from Miletić et al. (2021)
load('./data/data_exp1.RData')

# Exclude participants that are marked 'excl' by Miletic et al. (2021)
dat <- dat[!dat$excl, ]
dat <- dat[!is.na(dat$rt),]
dat$rt <- round(dat$rt, 3)  # round RTs to milliseconds
```

1.1 From data to a basic RL-EAM: the RL-RD

While *EMC2* is flexibly designed to allow any mapping between learning signals and decision parameters, most RL-EAM applications assume that drift rates are a function of Q-values. Implementing this requires four steps:

1. Specify the covariates in the data;
2. Apply the delta rule to each covariate;
3. Define the trial-wise mapping between covariates and each accumulator's drift rate;
4. Specify a feedback generator for posterior predictive simulation.

Figure 1 illustrates how these steps connect. We work through each in turn, using dataset 1 from Miletić et al. (2021).

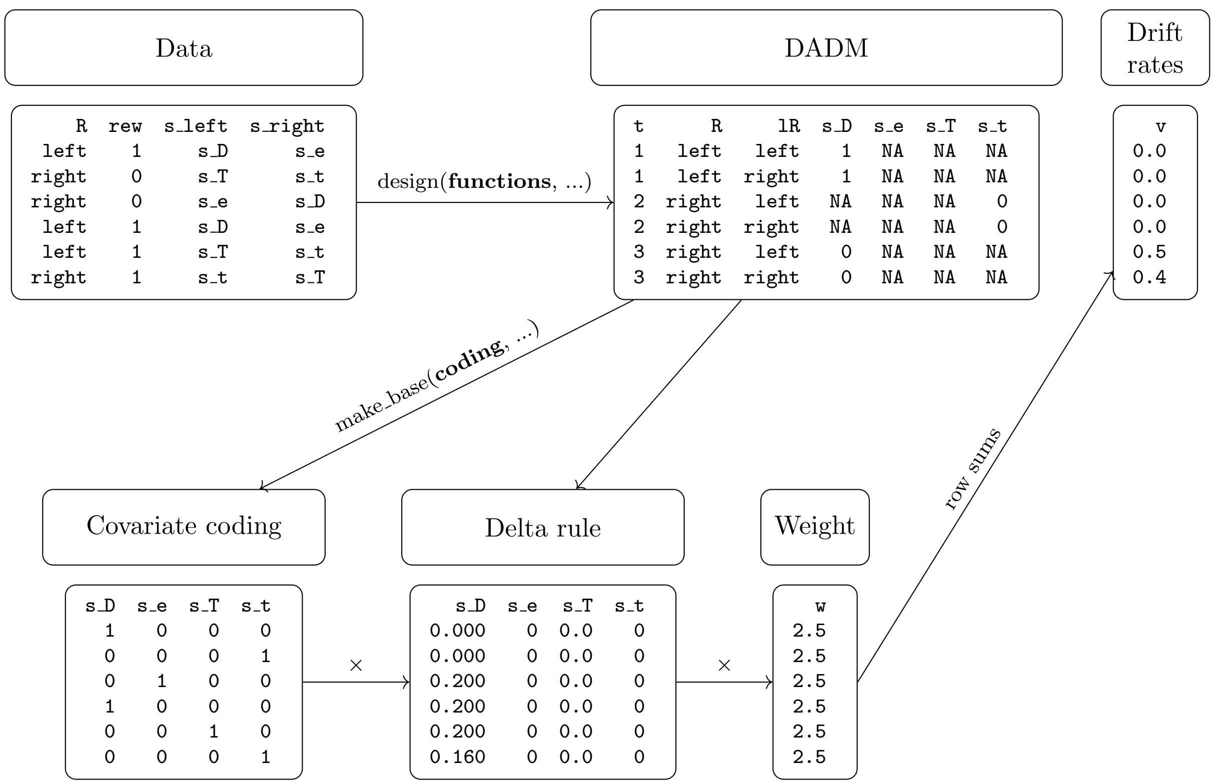


Figure 1: Overview of the steps required to implement an RL-RD in `EMC2`. Users specify two sets of functions: covariate functions supplied to `design()` that generate one column per symbol in the `dadm` (here, `s_D`, `s_e`, `s_T`, `s_t`); and a coding function supplied to `make_base()` that specifies which covariate drives which accumulator on each trial. The trial-wise drift rate is the row sum of the updated Q-values multiplied by the covariate coding and the weight parameter.

1.1.1 Step 1: Covariate specification

We begin by formatting the data into the structure *EMC2* requires for all choice tasks: a `subjects` column (participant identifier), `S` (stimulus), `R` (response), and `rt` (response time in seconds). `S` and `R` are response-coded — here, they refer to the direction of the chosen response (left/right). In this experiment, the optimal direction (i.e., the one with the higher reward probability) varies across trials, so we derive `S` from the reward probability columns:

```r
dat$subjects <- as.factor(dat$pp)
dat$R        <- factor(dat$choiceDirection, levels = c('left', 'right'))
dat$S        <- factor(ifelse(dat$p_win_left > dat$p_win_right, 'left', 'right'),
                       levels = c('left', 'right'))

print(head(dat[, c('subjects', 'S', 'R', 'rt')]))
```

```
#>      subjects     S     R    rt
#> 3121        7  left  left 0.750
#> 3122        7  left right 0.867
#> 3123        7 right right 1.100
#> 3124        7  left  left 0.708
#> 3125        7 right right 1.175
#> 3126        7  left  left 1.483
```

This is sufficient to fit a standard EAM. Instrumental learning tasks add two further pieces of information: the `reward` received on each trial, and the `symbol` to which that reward was assigned. Note that the reward does not have to correspond to the *chosen* symbol — experiments with counterfactual feedback (Palminteri et al. 2017) provide feedback for both symbols simultaneously. What matters is knowing *which* symbol each piece of feedback belongs to. We extract these columns next:

```r
# Recode reward to [0, 1]
dat$reward  <- dat$reward / 100

# Symbol presented on the left and right on each trial
dat$s_left  <- paste0('s_', dat$appear_left)
dat$s_right <- paste0('s_', dat$appear_right)

# We also keep track of the 'exposure' - i.e., the number of times a stimulus pair has been
presented. This is not required for fitting, but facilitates plotting later on.
dat$exposure <- get_exposure(dat)

print(head(dat[, c('subjects', 'S', 'R', 'rt', 'reward', 's_left', 's_right', 'exposure')]))
```

```
#>      subjects     S     R    rt reward s_left s_right exposure
#> 3121        7  left  left 0.750      1    s_D     s_e        1
#> 3122        7  left right 0.867      0    s_T     s_t        1
#> 3123        7 right right 1.100      0    s_e     s_D        2
#> 3124        7  left  left 0.708      0    s_J     s_h        1
#> 3125        7 right right 1.175      1    s_k     s_K        1
#> 3126        7  left  left 1.483      1    s_J     s_h        2
```

For posterior predictive simulation we also need to be able to regenerate feedback in the same way it was generated in the experiment — here, by sampling from a Bernoulli distribution with symbol-specific reward probabilities. We therefore retain those probabilities, and trim the data frame to the columns needed for modelling:

```r
dat$p_left  <- dat$p_win_left
dat$p_right <- dat$p_win_right

dat <- dat[, c('subjects', 'S', 'R', 'rt', 'reward',
               's_left', 's_right', 'p_left', 'p_right', 'exposure')]
print(head(dat))
```

```
#>      subjects     S     R    rt reward s_left s_right p_left p_right exposure
#> 3121        7  left  left 0.750      1    s_D     s_e   0.70    0.30        1
#> 3122        7  left right 0.867      0    s_T     s_t   0.80    0.20        1
#> 3123        7 right right 1.100      0    s_e     s_D   0.30    0.70        2
#> 3124        7  left  left 0.708      0    s_J     s_h   0.65    0.35        1
#> 3125        7 right right 1.175      1    s_k     s_K   0.40    0.60        1
#> 3126        7  left  left 1.483      1    s_J     s_h   0.65    0.35        2
```

Note that the column names `reward`, `s_left`, `s_right`, `p_left`, and `p_right` are arbitrary — as long as the user-defined functions described below refer to the correct columns, any names will work.

The next step is to create one column per symbol, indicating the reward received when that symbol was chosen (and `NA` otherwise). An obvious approach would be to add these columns directly to the data. However, *EMC2* treats all non-`rt`, non-`R` columns as fixed design factors. This is a problem for simulation: when generating posterior predictives, the covariate columns must be recomputed from the *simulated* choices and rewards, not copied from the observed data. We therefore define *functions* that construct the covariate columns on the fly. *EMC2* re-applies these functions after each simulated trial, ensuring that Q-values track simulated rather than observed behaviour:

```r
# Generator: returns a function that extracts the reward for one specific symbol
covariate_column_generator <- function(col_name) {
  f <- function(data) {
    RS <- ifelse(data$R == 'left', data$s_left, data$s_right)
    out <- rep(NA, nrow(data))
    out[(RS == col_name)] <- data$reward[(RS == col_name)]
    out
  }
  return(f)
}

all_symbols <- unique(c(dat$s_left, dat$s_right))
make_covariate_columns <- setNames(
  lapply(all_symbols, covariate_column_generator),
  all_symbols
)
```

`make_covariate_columns` is now a named list of functions, one per symbol. We pass it to `design()` via the `functions` argument, which tells *EMC2* to apply them to the data-augmented design matrix (`dadm`) at construction time — and again during simulation:

```r
design_RDM <- design(model          = RDM,
                     data           = dat,
                     functions      = make_covariate_columns,
                     matchfun       = function(d) d$S == d$lR,
                     formula        = list(B ~ 1, v ~ 1, t0 ~ 1),
                     report_p_vector = FALSE)
```

```r
emc <- make_emc(dat, design_RDM)
```

```r
print(head(emc[[1]]$data[[1]]))
```

```
#>   subjects     S     R        rt reward s_left s_right p_left p_right exposure
#> 1        7  left  left 0.7500000      1    s_D     s_e    0.7     0.3        1
#> 2        7  left  left 0.7500000      1    s_D     s_e    0.7     0.3        1
```

```
#> 3        7  left right 0.8666667      0    s_T     s_t    0.8     0.2        1
#> 4        7  left right 0.8666667      0    s_T     s_t    0.8     0.2        1
#> 5        7 right right 1.1000000      0    s_e     s_D    0.3     0.7        2
#> 6        7 right right 1.1000000      0    s_e     s_D    0.3     0.7        2
#>   trials    lR    lM winner s_D s_T s_e s_J s_k s_h s_t s_K
#> 1      1  left  TRUE   TRUE   1  NA  NA  NA  NA  NA  NA  NA
#> 2      1 right FALSE  FALSE   1  NA  NA  NA  NA  NA  NA  NA
#> 3      2  left  TRUE  FALSE  NA  NA  NA  NA  NA  NA   0  NA
#> 4      2 right FALSE   TRUE  NA  NA  NA  NA  NA  NA   0  NA
#> 5      3  left FALSE  FALSE   0  NA  NA  NA  NA  NA  NA  NA
#> 6      3 right  TRUE   TRUE   0  NA  NA  NA  NA  NA  NA  NA
```

The symbol columns are now present in the `dadm`, with `NA` wherever a symbol was not chosen on that trial.

1.1.2 Step 2: Apply the delta rule

Trialwise covariates are handled through `trend` objects in *EMC2*, exactly as in the dynamic EAM tutorial. The kernel specifies the update rule applied to the covariate — here, the delta rule — and the base specifies which decision parameter is informed by the result and how. Since we want Q-values to influence drift rates linearly on the natural scale, we use a `lin` base with `phase = "posttransform"`:

```
delta_kernel <- make_kernel(cov_names = all_symbols, type = 'delta')
lin_base     <- make_base(target_parameter = 'v',
                          type             = 'lin',
                          kernel           = delta_kernel,
                          phase            = 'posttransform')
trend <- make_trend(lin_base)
```

1.1.3 Step 3: Define the accumulator coding

By default, *EMC2* sums the updated Q-values across all symbols when computing drift rates. On each trial, however, only two symbols are shown, and each accumulator should be driven only by the Q-value of the symbol it represents. We therefore define a *coding* function that returns a matrix with one row per `dadm` row and one column per symbol, with a 1 where the symbol matches the accumulator's response option and 0 elsewhere:

```
make_RL_RD_coding <- function(dadm, cov_names) {
  lS <- ifelse(dadm$lR == 'left', dadm$s_left, dadm$s_right)
  sapply(cov_names, function(col) as.numeric(lS == col))
}

print(head(make_RL_RD_coding(emc[[1]]$data[[1]], all_symbols)))
```

```
#>      s_D s_T s_e s_J s_k s_h s_t s_K
#> [1,]   1   0   0   0   0   0   0   0
#> [2,]   0   0   1   0   0   0   0   0
#> [3,]   0   1   0   0   0   0   0   0
#> [4,]   0   0   0   0   0   0   1   0
#> [5,]   0   0   1   0   0   0   0   0
#> [6,]   1   0   0   0   0   0   0   0
```

We pass this to `make_base()` via the `coding` argument, rebuild the trend and design, and fix `v.q0 = 0` (initial Q-values are hard to estimate):

```r
lin_base <- make_base(target_parameter = 'v',
                      type             = 'lin',
                      kernel           = delta_kernel,
                      phase            = 'posttransform',
                      coding           = list(w = make_RL_RD_coding))
trend <- make_trend(lin_base)

design_RDM <- design(model           = RDM,
                     data            = dat,
                     functions       = make_covariate_columns,
                     matchfun        = function(d) d$S == d$lR,
                     formula         = list(B ~ 1, v ~ 1, t0 ~ 1),
                     trend           = trend,
                     constants       = c('v.q0' = 0),
                     report_p_vector = FALSE)
```

```r
emc <- make_emc(dat, design_RDM)
```

The drift rate for each accumulator is now determined solely by the Q-value of the symbol that accumulator represents.

1.1.4 Step 4: Feedback generator

The feedback generator is a function that *EMC2* calls during simulation to produce trial-by-trial rewards from simulated choices. It receives the `dadm` (with simulated responses in `R`) and must return a reward vector of the same length. Here, we sample from a Bernoulli distribution using the reward probability of the chosen symbol. One subtlety is that race models such as the RDM have one row per accumulator per trial in the dadm, whereas the DDM has one row per trial. We therefore define a single unified feedback generator that detects the dadm structure and acts accordingly — this function can be reused across all models in this tutorial:

```r
feedback_generator <- function(d) {
  if ("lR" %in% names(d) && nlevels(d$lR) > 1) {
    # Race model: dadm has one row per accumulator per trial.
    # Subset to one row per trial, generate rewards, then repeat
    # to match the full dadm length.
    data      <- d[d$lR == levels(d$lR)[1], ]
    p_chosen <- ifelse(data$R == 'left', data$p_left, data$p_right)
    rep(rbinom(nrow(data), 1, p_chosen), each = nlevels(d$lR))
  } else {
    # DDM: dadm has one row per trial - generate rewards directly.
    p_chosen <- ifelse(d$R == 'left', d$p_left, d$p_right)
    rbinom(nrow(d), 1, p_chosen)
  }
}

design_RDM <- design(model            = RDM,
                     data             = dat,
                     functions        = c(reward=feedback_generator,
                                          make_covariate_columns),
                     matchfun         = function(d) d$S == d$lR,
                     formula          = list(B ~ 1, v ~ 1, t0 ~ 1),
                     trend            = trend,
                     constants        = c('v.q0' = 0),
                     report_p_vector = FALSE)
```

```r
emc <- make_emc(dat, design_RDM)
```

1.1.5 Fitting and checking the RL-RD

With the covariates, kernel, coding, and feedback generator specified, we can now estimate the RL-RD model.

```r
emc_rlrd <- fit(emc)
```

After fitting, it is important to check convergence. The `check()` function plots the group-level mean, group-level variance, and subject-level parameters with the highest R-hat (i.e., worst convergence), and prints convergence statistics (Rhat and effective sample size) to the console.

```r
check(emc_rlrd)
```

```
#> Iterations:
#>      preburn burn adapt sample
#> [1,]       0    0     0   1000
#> [2,]       0    0     0   1000
#> [3,]       0    0     0   1000
#>
#>  mu
#>             B        v      t0  v.alpha    v.w_w
#> Rhat    1.009    1.004   1.005    1.004    1.002
#> ESS  1094.000 1473.000 792.000 1906.000 1950.000


#>
#>  sigma2
```

```
#>             B        v      t0  v.alpha    v.w_w
#> Rhat   1.003    1.009   1.006    1.003    1.003
#> ESS   949.000 1109.000 848.000 1303.000 1449.000

#>
#>  alpha highest Rhat : 14
#>             B       v      t0 v.alpha   v.w_w
#> Rhat   1.013   1.008   1.015   1.028   1.027
#> ESS   708.000 869.000 806.000 813.000 910.000
```

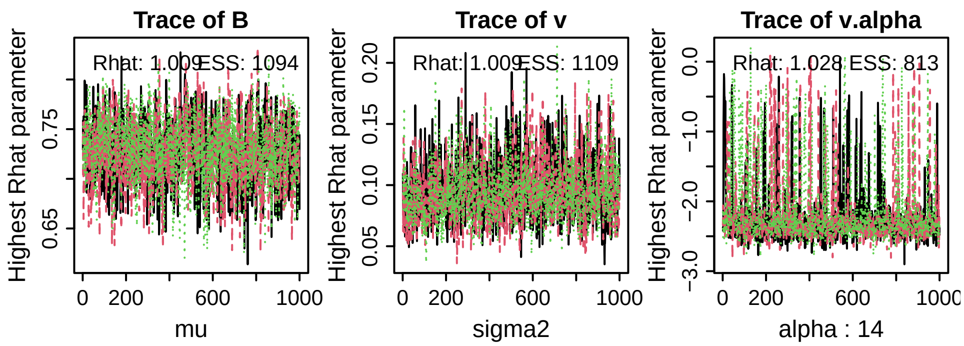


Figure 2: Trace plots of the sampled parameters with highest Rhat values (i.e., worst convergence) at the group-level mean (left, mu), the group-level variance (middle, sigma2), and the subject-level (alpha; here, subject 8). ESS = effective sample size.

1.1.6 Posterior summaries and predictive checks

Superimposed prior and posterior distributions and credible intervals can be obtained with `plot_pars()` and `credint()`, respectively. Both accept a `selection` argument specifying which type of parameters to use — by default `"mu"` (group-level means).

```
plot_pars(emc_rlrd)
```

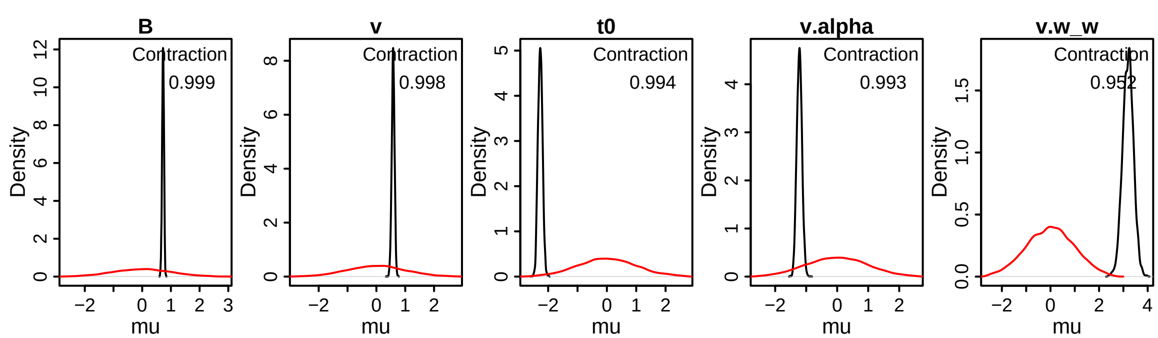


Figure 3: Posterior density (black) and prior density (red) of sampled group-level mean parameters of the RL-RD. Contraction indicates the reduction of uncertainty in the posterior versus the prior (1 minus the ratio of the posterior to prior variance).

```
credint(emc_rlrd)
```

```
#> $mu
#>            2.5%    50%  97.5%
#> B         0.661  0.727  0.787
#> v         0.482  0.581  0.673
#> t0       -2.409 -2.270 -2.120
#> v.alpha -1.375 -1.219 -1.053
#> v.w_w    2.755  3.196  3.613
```

Supplying the fitted model to the `predict()` function generates posterior predictive datasets, which can then be used to assess global quality of fit. Here, we use a custom plotting function `plot_learning()` from the accompanying utilities file to summarise accuracy and response time distributions. We first determine the number of times a stimulus has been presented (the exposure) in both the data and the posterior predictives, which we then aggregate into 10 bins to reduce noise.

```
pp_rlrd <- predict(emc_rlrd)

plot_learning(dat=dat_rlrd, pp=pp_rlrd, x.var='exposure', n.breaks=10)
```

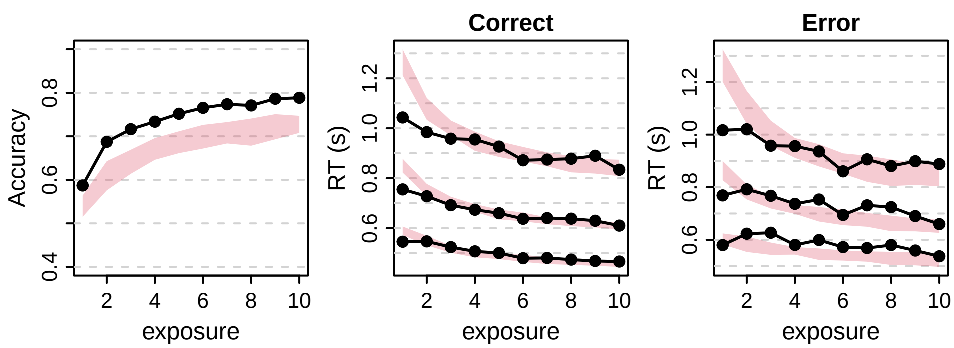


Figure 4: Learning plots of the RL-RD. Left panel shows accuracy over exposure (i.e., the number of times a stimulus pair has been seen), binned into 10 bins. Black lines are data, red shaded areas are 90% credible interval of the model's posterior predictives. Middle and right panels show correct and error RT distributions, respectively. The lower, middle, and top lines correspond to the 10th, 50th, and 90th RT quantiles.

1.2 Changing the EAM: RL-DDM

A popular alternative to the RL-RD is the RL-DDM, which replaces the racing diffusion process with a Wiener diffusion model. The key difference lies in the accumulator coding. In the RL-RD, each accumulator is driven by the Q-value of the symbol it represents: the left accumulator by Q_{left}, the right accumulator by Q_{right}. In the DDM, there is a single drift rate that reflects the *relative* evidence for one response over the other. The natural RL analogue is therefore a difference coding: $v = w \cdot (Q_{\text{right}} - Q_{\text{left}})$, where positive values push the process towards the upper (right) boundary and negative values towards the lower (left) boundary. This is implemented by replacing the accumulator-specific coding of the RL-RD with a difference coding function:

```
make_RL_DDM_coding <- function(dadm, cov_names) {
  coding_left  <- sapply(cov_names, function(col) ifelse(dadm$s_left  == col,  1, 0))
  coding_right <- sapply(cov_names, function(col) ifelse(dadm$s_right == col,  1, 0))
  coding_right - coding_left
}
```

For a given trial, this returns +1 for the symbol on the right, −1 for the symbol on the left, and 0 for all other symbols. The row sum of the updated Q-values multiplied by this coding therefore is $Q_{\text{right}} - Q_{\text{left}}$, which is then scaled by the weight parameter w to obtain the drift rate. Everything else — the delta kernel, the `lin` base, the feedback generator — carries over unchanged:

```
delta_kernel_ddm <- make_kernel(cov_names = all_symbols, type = 'delta')
lin_base_ddm     <- make_base(target_parameter = 'v',
                              type             = 'lin',
                              kernel           = delta_kernel_ddm,
                              phase            = 'posttransform',
                              coding           = list(w = make_RL_DDM_coding))
trend_ddm <- make_trend(lin_base_ddm)

design_DDM <- design(model            = DDM,
                     data             = dat,
                     functions        = c(reward=feedback_generator,
                                          make_covariate_columns),
                     matchfun         = function(d) d$S == d$lR,
                     formula          = list(a ~ 1, v ~ 1, t0 ~ 1),
                     trend            = trend_ddm,
                     constants        = c('v.q0' = 0),
                     report_p_vector = FALSE)
```

```
emc_rlddm <- make_emc(dat, design_DDM)
```

```
emc_rlddm <- fit(emc_rlddm)
```

The posterior predictives reveal that the RL-DDM does not fit the shape of the response time distributions well:

```
pp_rlddm <- predict(emc_rlddm)

plot_learning(dat=dat, pp=pp_rlddm, x.var='exposure')
```

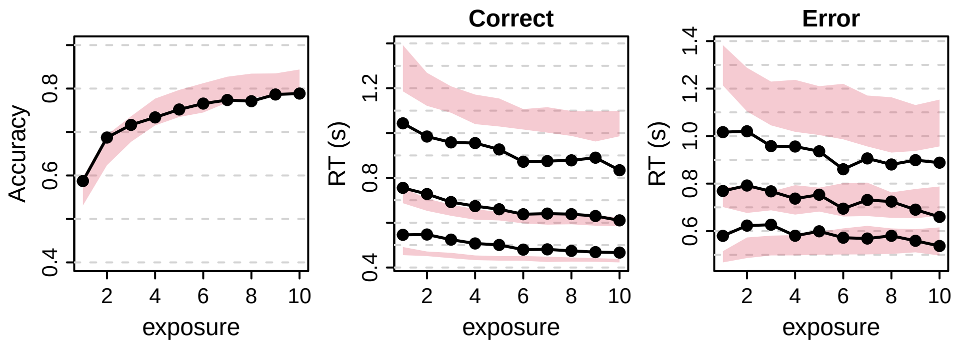


Figure 5: Learning plots of the RL-DDM. Left panel shows accuracy over exposure (i.e., the number of times a stimulus pair has been seen), binned into 10 bins. Black lines are data, red shaded areas are 90% credible interval of the model's posterior predictives. Middle and right panels show correct and error RT distributions, respectively. The lower, middle, and top lines correspond to the 10th, 50th, and 90th RT quantiles.

1.3 Changing the EAM: RL-ARD

The RL-ARD (Miletić et al. 2021) combines the race architecture of the RL-RD with a richer parameterisation of how Q-values map to drift rates. To see why this is useful, consider what the RL-RD and RL-DDM codings each capture. The RL-RD coding maps each Q-value directly to the drift rate of the corresponding accumulator: the left accumulator races at a rate proportional to Q_{left}, the right at a rate proportional to Q_{right}. The RL-DDM coding maps the *difference* $Q_{\text{right}} - Q_{\text{left}}$ to a single drift rate, capturing only the relative preference between options. Neither coding alone is fully satisfactory: the RL-RD cannot represent relative preference independently of overall evidence strength, and the RL-DDM discards the absolute level of Q-values entirely.

The RL-ARD resolves this by decomposing the Q-values into two components, each mapped to drift rates via a separate base:

- A **difference** component, $Q_{\text{self}} - Q_{\text{other}}$, weighted by w_d. This captures the relative preference for the accumulator's own option over the alternative — analogous to the RL-DDM drift rate, but applied within a race architecture.
- A **sum** component, $Q_{\text{self}} + Q_{\text{other}}$, weighted by w_s. This captures the overall level of evidence, regardless of which option is preferred. A high sum means both options are well-learned; a low sum means both are uncertain.

Together, the drift rate for accumulator i is:

$$v_i = v_0 + w_d \cdot (Q_i - Q_j) + w_s \cdot (Q_i + Q_j)$$

where v_0 is the baseline drift rate intercept (sometimes interpreted as an urgency signal) and j indexes the competing accumulator. This parameterisation allows the data to determine the relative contribution of absolute and relative Q-values to response speed and accuracy.

1.3.1 Coding functions

The difference and sum codings extend the earlier examples in a straightforward way. Both require knowing not only which symbol the accumulator represents (`lS`, as in the RL-RD coding) but also which symbol the *competing* accumulator represents (`lSother`):

```
make_RL_ARD_difference <- function(dadm, cov_names) {
  lS      <- ifelse(dadm$lR == 'left', dadm$s_left,  dadm$s_right)
  lSother <- ifelse(dadm$lR == 'left', dadm$s_right, dadm$s_left)
  # +1 for own symbol, -1 for competing symbol (cf. RL-DDM: +1 right, -1 left)
  codinglS      <- sapply(cov_names, function(col) as.numeric(lS      == col))
  codinglSother <- sapply(cov_names, function(col) as.numeric(lSother == col))
  codinglS - codinglSother
}

make_RL_ARD_sum <- function(dadm, cov_names) {
  lS      <- ifelse(dadm$lR == 'left', dadm$s_left,  dadm$s_right)
  lSother <- ifelse(dadm$lR == 'left', dadm$s_right, dadm$s_left)
  # +1 for both own and competing symbol
  codinglS      <- sapply(cov_names, function(col) as.numeric(lS      == col))
  codinglSother <- sapply(cov_names, function(col) as.numeric(lSother == col))
  codinglS + codinglSother
}
```

Note the relationship to the earlier codings: `make_RL_ARD_difference` generalises `make_RL_DDM_coding` from a fixed left/right frame to an accumulator-relative frame (own minus other, rather than right minus left). `make_RL_ARD_sum` is new, and has no direct analogue in the RL-RD or RL-DDM.

1.3.2 Design specification

Each coding function is passed to its own `make_base()` call, and the two bases are combined into a single trend with `make_trend()`. The delta kernel and feedback generator are shared with the earlier models:

```
delta_kernel_ard <- make_kernel(cov_names = all_symbols, type = 'delta')

base_diff <- make_base(target_parameter = 'v',
                       type             = 'lin',
                       kernel           = delta_kernel_ard,
                       phase            = 'posttransform',
                       coding           = list(d = make_RL_ARD_difference))
base_sum  <- make_base(target_parameter = 'v',
                       type             = 'lin',
                       kernel           = delta_kernel_ard,
                       phase            = 'posttransform',
                       coding           = list(s = make_RL_ARD_sum))
trend_ard <- make_trend(base_diff, base_sum)

design_ARD <- design(model           = RDM,
                     data            = dat,
                     functions       = c(reward=feedback_generator,
                                         make_covariate_columns),
                     matchfun        = function(d) d$S == d$lR,
                     formula         = list(B ~ 1, v ~ 1, t0 ~ 1),
                     trend           = trend_ard,
                     constants       = c('v.q0' = 0),
                     report_p_vector = FALSE)
```

```
emc_rlard <- make_emc(dat, design_ARD)
```

The RL-ARD introduces one additional sampled parameter compared to the RL-RD: the sum weight `v.w_s` (alongside the difference weight `v.w_d`, the learning rate `v.alpha`, the baseline drift `v`, the threshold `B`, and the non-decision time `t0`).

1.3.3 Fitting and convergence

```
emc_rlard <- fit(emc_rlard)
check(emc_rlard)
```

```
#> Iterations:
#>      preburn burn adapt sample
#> [1,]       0    0     0   1000
#> [2,]       0    0     0   1000
#> [3,]       0    0     0   1000
#>
#>  mu
#>             B     v     t0  v.alpha    v.w_d    v.w_s
#> Rhat    1.001     1  1.001    1.004    1.006    1.004
```

```
#> ESS  1014.000 1575 838.000 2292.000 2001.000 2385.000


#>
#>  sigma2
#>         B         v       t0 v.alpha     v.w_d     v.w_s
#> Rhat    1     1.001    1.003       1     1.003     1.001
#> ESS   820 1655.000 844.000     1581 1585.000 1540.000


#>
#>  alpha highest Rhat : 33
#>              B         v        t0  v.alpha v.w_d     v.w_s
#> Rhat     1.002     1.025     1.001     1.002     1     1.001
#> ESS   1475.000 2328.000 1420.000 2873.000  2588 2452.000
```

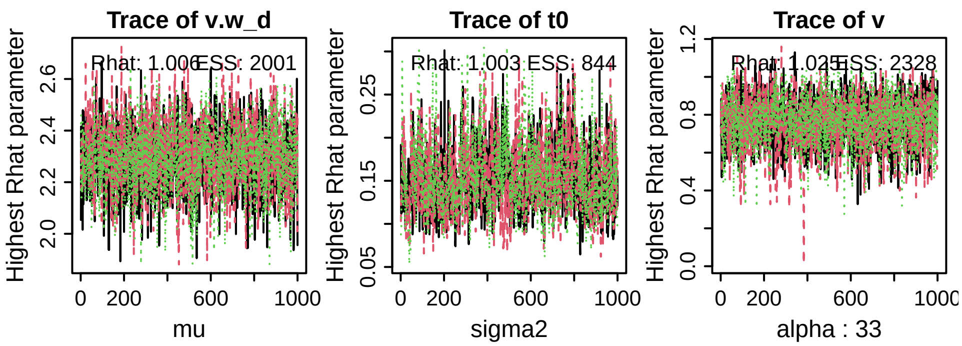


Figure 6: Trace plots of the sampled parameters with highest Rhat values (i.e., worst convergence) at the group-level mean (left, mu), the group-level variance (middle, sigma2), and the subject-level (alpha; here, subject 25). ESS = effective sample size.

1.3.4 Posteriors

```
plot_pars(emc_rlard)
```

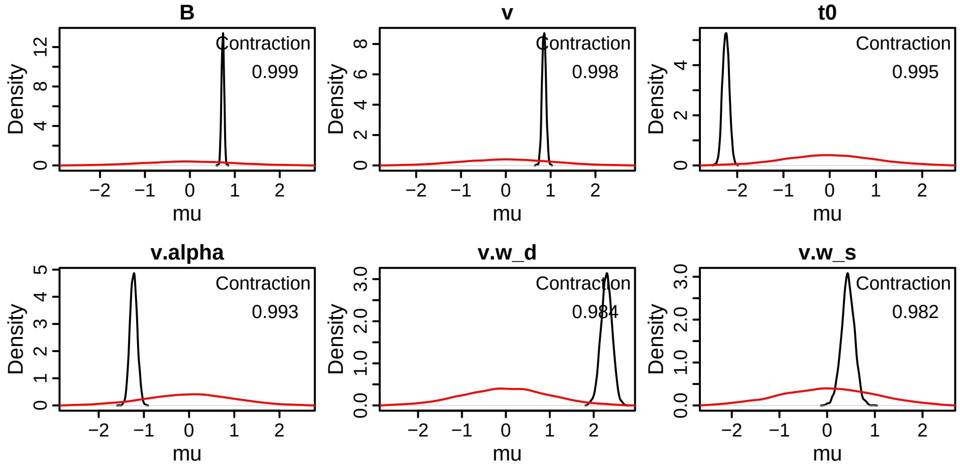


Figure 7: Posterior density (black) and prior density (red) of sampled group-level mean parameters of the RL-ARD. `v.alpha` is the learning rate; `v.w_d` and `v.w_s` are the difference and sum weights, respectively; `v` is the baseline drift rate; `B` is the threshold; and `t0` is the non-decision time. Contraction indicates the reduction of uncertainty in the posterior versus the prior.

```
credint(emc_rlard)
```

```
#> $mu
#>            2.5%    50%  97.5%
#> B         0.675  0.733  0.789
#> v         0.765  0.853  0.939
#> t0       -2.385 -2.246 -2.107
#> v.alpha  -1.379 -1.228 -1.068
#> v.w_d     2.037  2.294  2.536
#> v.w_s     0.170  0.436  0.698
```

1.3.5 Posterior predictives

```
pp_rlard <- predict(emc_rlard, return_trialwise_parameters = TRUE)

plot_learning(dat=dat, pp=pp_rlard, x.var='exposure')
```

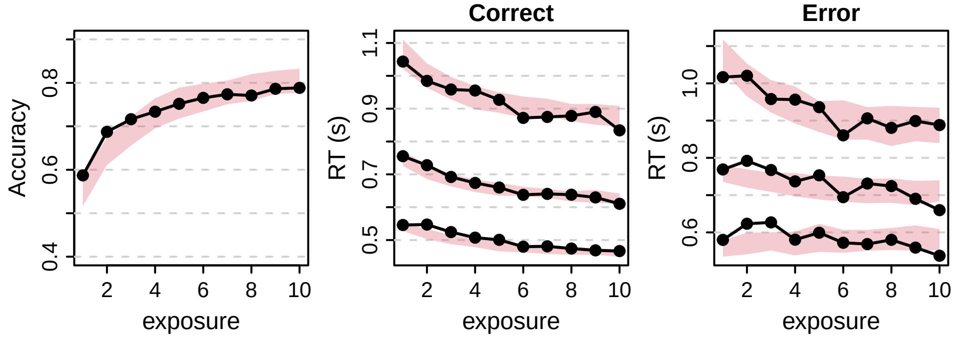


Figure 8: Learning plots of the RL-ARD. Left panel shows accuracy over exposure (i.e., the number of times a stimulus pair has been seen), binned into 10 bins. Black lines are data, red shaded areas are 90% credible interval of the model's posterior predictives. Middle and right panels show correct and error RT distributions, respectively. The lower, middle, and top lines correspond to the 10th, 50th, and 90th RT quantiles.

1.3.6 Model comparison

We can now formally compare the three models using `compare()`, which returns DIC, BPIC, and marginal deviance (MD $= -2\times$ marginal likelihood); lower values indicate a preferred model:

```
compare(list(rlrd  = emc_rlrd,
             rlddm = emc_rlddm,
             rlard = emc_rlard))
```

```
#>         MD wMD  DIC wDIC BPIC wBPIC EffectiveN meanD Dmean minD
#> rlrd  6048   0 5616    0 5922     0        306  5310  5142 5004
#> rlddm 8159   0 7631    0 7930     0        299  7332  7094 7034
#> rlard 4977   1 4517    1 4892     1        376  4141  3910 3765
```

2 Factors on parameters — the speed-accuracy trade-off

Many research questions ask which cognitive processes are affected by an experimental manipulation. A classic example in the decision-making literature is the speed-accuracy trade-off (SAT): when instructed to respond quickly, people tend to lower their response thresholds; when instructed to be accurate, they raise them. This section reproduces the results from experiment 2 of Miletić et al. (2021), in which participants performed an instrumental learning task with trial-by-trial cues instructing them to emphasise either speed or accuracy. We fit an RL-ARD to test whether this manipulation affected response thresholds (`B`), drift rate urgency (`v`), or both.

2.0.1 Data

The data preparation follows the same steps as Section 1.1, and retains the `cue` column (levels: `"accuracy"`, `"speed"`), which will enter the model formula as a between-condition factor:

```r
load('./data/data_exp3.RData')
dat <- dat[!dat$excl, ]
dat <- dat[!is.na(dat$rt), ]
dat$rt <- round(dat$rt, 3)

dat$subjects <- as.factor(dat$pp)
dat$R        <- factor(dat$choice_direction,
                       levels = c(0, 1), labels = c('left', 'right'))
dat$S        <- factor(ifelse(dat$p_win_left > dat$p_win_right, 'left', 'right'),
                       levels = c('left', 'right'))
dat$reward   <- dat$outcome
dat$s_left   <- paste0('s_', dat$stimulus_symbol_left)
dat$s_right  <- paste0('s_', dat$stimulus_symbol_right)
dat$p_left   <- dat$p_win_left
dat$p_right  <- dat$p_win_right
dat$cue      <- as.factor(dat$cue)

dat$exposure <- get_exposure(dat)

dat <- dat[, c('subjects', 'S', 'R', 'rt', 'reward', 's_left', 's_right', 'p_left', 'p_right',
'cue', 'exposure')]

print(head(dat))
```

```
#>    subjects     S     R    rt reward s_left s_right p_left p_right cue exposure
#> 6         1  left right 0.492      0    s_C     s_c    0.8     0.2 SPD        1
#> 19        1  left  left 0.709      1    s_D     s_g    0.7     0.3 ACC        1
#> 32        1  left right 0.492      0    s_f     s_x    0.6     0.4 SPD        1
#> 45        1  left  left 0.601      1    s_f     s_x    0.6     0.4 ACC        2
#> 58        1 right right 0.685      1    s_g     s_D    0.3     0.7 ACC        2
#> 71        1  left  left 0.459      1    s_f     s_x    0.6     0.4 SPD        3
```

```r
all_symbols_sat <- unique(c(dat$s_left, dat$s_right))
make_covariate_columns_sat <- setNames(
  lapply(all_symbols_sat, covariate_column_generator),
  all_symbols_sat
)
```

The RL component is identical to the RL-ARD in Section 1.3: a delta kernel with difference and sum bases, and the feedback generator. The only change is in the model formula, where `B` and `v` are allowed to vary with `cue`. We use a single-column contrast matrix `ADmat` that codes the accuracy-minus-speed difference, so the estimated effect directly reflects the change in each parameter when moving from the speed to the accuracy cue:

```r
kernel_sat <- make_kernel(all_symbols_sat, 'delta')
base_diff_sat <- make_base('v', 'lin', kernel = kernel_sat,
                           coding = list(d = make_RL_ARD_difference),
                           phase  = 'posttransform')
base_sum_sat  <- make_base('v', 'lin', kernel = kernel_sat,
                           coding = list(s = make_RL_ARD_sum),
                           phase  = 'posttransform')
trend_sat <- make_trend(base_diff_sat, base_sum_sat)

# Contrast matrix: accuracy - speed
ADmat <- matrix(c(1, -1), ncol = 1,
                dimnames = list(NULL, c('a-s')))

design_sat_full <- design(model       = RDM,
                          data        = dat,
                          functions   = c(reward=feedback_generator,
                                          make_covariate_columns_sat),
                          matchfun    = function(d) d$S == d$lR,
                          contrasts   = list(cue = ADmat),
                          formula     = list(B ~ cue, v ~ cue, t0 ~ 1),
                          trend       = trend_sat,
                          constants   = c('v.q0' = 0),
                          report_p_vector = FALSE)
```

We also specify a threshold-only model, in which v is fixed across cues, to test whether the SAT manipulation affected drift rates over and above thresholds:

```r
design_sat_B <- design(model       = RDM,
                       data        = dat,
                       functions   = c(reward=feedback_generator,
                                       make_covariate_columns_sat),
                       matchfun    = function(d) d$S == d$lR,
                       contrasts   = list(cue = ADmat),
                       formula     = list(B ~ cue, v ~ 1, t0 ~ 1),
                       trend       = trend_sat,
                       constants   = c('v.q0' = 0),
                       report_p_vector = FALSE)
```

2.0.2 Fitting

```r
emc_sat_full <- make_emc(dat, design_sat_full)
emc_sat_full <- fit(emc_sat_full)

emc_sat_B <- make_emc(dat, design_sat_B)
emc_sat_B <- fit(emc_sat_B)
```

2.0.3 Convergence and posteriors

```r
check(emc_sat_full)
```

```
#> Iterations:
#>      preburn burn adapt sample
```

```
#> [1,]       0    0     0   1000
#> [2,]       0    0     0   1000
#> [3,]       0    0     0   1000
#>
#>  mu
#>             B B_cuea-s        v v_cuea-s       t0  v.alpha    v.w_d    v.w_s
#> Rhat    1.001        1    1.001        1    1.001    1.005    1.001    1.007
#> ESS  2065.000     2700 2241.000     1830 2150.000 1350.000 1102.000 1320.000


#>
#>  sigma2
#>             B B_cuea-s        v v_cuea-s       t0 v.alpha    v.w_d   v.w_s
#> Rhat    1.004    1.003    1.002    1.001    1.002   1.014    1.004   1.008
#> ESS  1291.000 1938.000 1325.000 1016.000 1949.000 727.000 1123.000 836.000


#>
#>  alpha highest Rhat : 2
#>             B B_cuea-s        v v_cuea-s       t0 v.alpha   v.w_d    v.w_s
#> Rhat    1.001        1    1.003    1.001    1.001   1.014   1.009    1.008
#> ESS  1725.000     1686 2063.000 1757.000 1654.000 732.000 575.000 1050.000
```

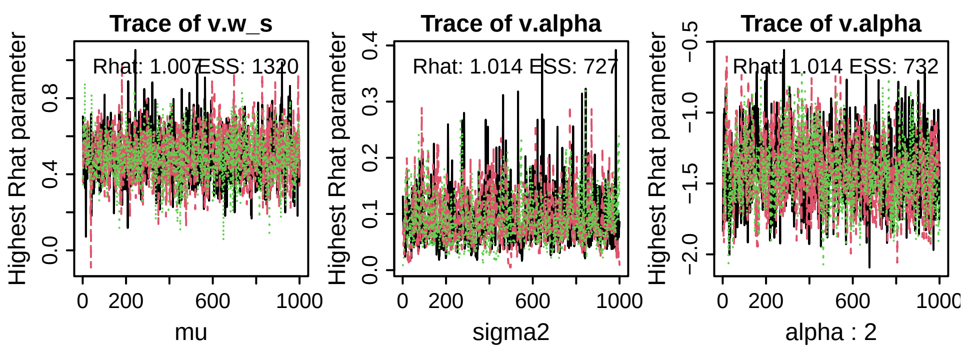


Figure 9: Trace plots of the sampled parameters with highest Rhat values (i.e., worst convergence) at the group-level mean (left, mu), the group-level variance (middle, sigma2), and the subject-level (alpha; here, subject 2). ESS = effective sample size.

```
plot_pars(emc_sat_full)
```

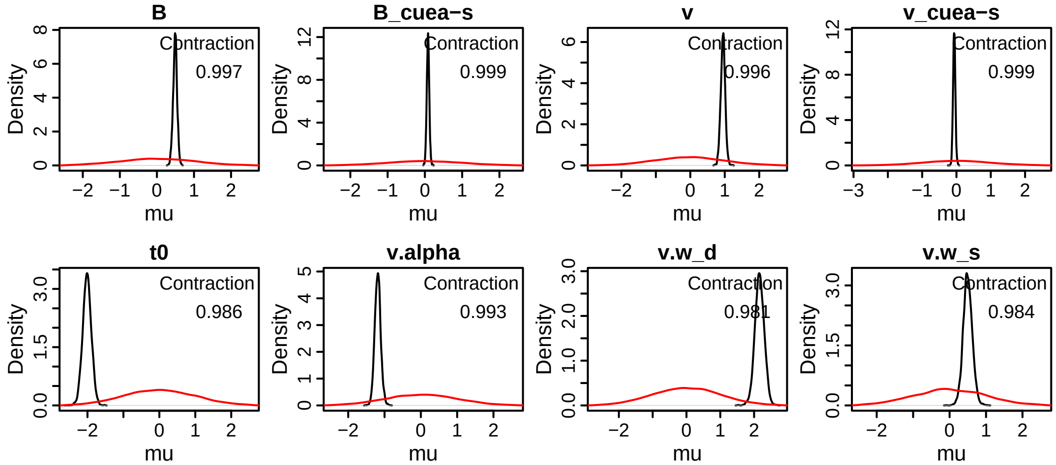


Figure 10: Posterior density (black) and prior density (red) of sampled group-level mean parameters of the full SAT model (`B  cue`, `v  cue`). Parameters suffixed `a-s` reflect the accuracy-minus-speed contrast. Positive values indicate a higher parameter estimate in the accuracy than the speed condition.

```
credint(emc_sat_full)
```

```
#> $mu
#>             2.5%    50%  97.5%
#> B          0.380  0.491  0.596
#> B_cuea-s   0.019  0.085  0.152
#> v          0.818  0.949  1.076
#> v_cuea-s  -0.131 -0.066  0.004
#> t0        -2.246 -2.012 -1.778
#> v.alpha   -1.345 -1.187 -1.016
#> v.w_d      1.891  2.160  2.424
#> v.w_s      0.258  0.496  0.745
```

```
# credint(emc_sat_full, map = TRUE)
```

2.0.4 Posterior predictives

```
pp_sat_full <- predict(emc_sat_full)

plot_learning(dat=dat, pp=pp_sat_full[is.finite(pp_sat_full$rt),], x.var='exposure', row.factor
= 'cue')
```

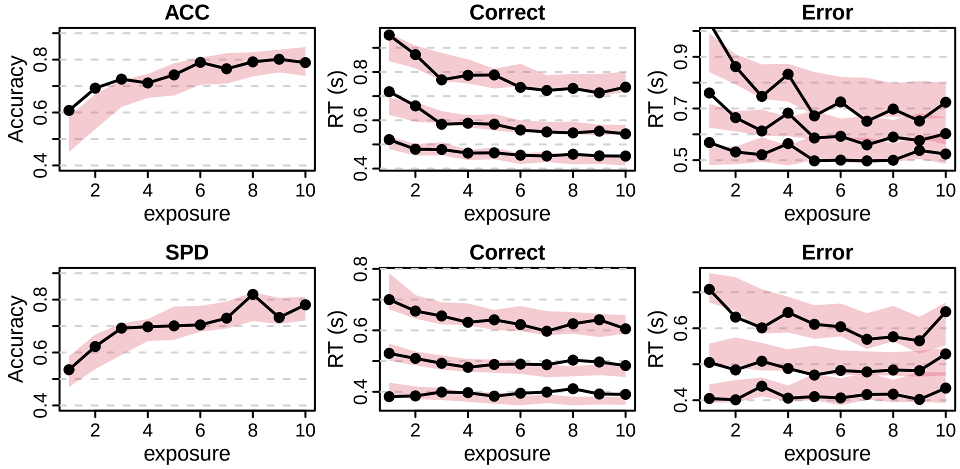


Figure 11: Learning plots of the full SAT RL-ARD, separately for each condition (rows). Left panel shows accuracy over exposure (i.e., the number of times a stimulus pair has been seen), binned into 10 bins. Black lines are data, red shaded areas are 90% credible interval of the model's posterior predictives. Middle and right panels show correct and error RT distributions, respectively. The lower, middle, and top lines correspond to the 10th, 50th, and 90th RT quantiles.

2.0.5 Model comparison

To test whether the cue affected drift rates over and above thresholds, we compare the full model against the threshold-only model:

```
compare(list(full = emc_sat_full,
             B    = emc_sat_B))
```

```
#>        MD wMD   DIC wDIC BPIC wBPIC EffectiveN meanD Dmean  minD
#> full -868   1 -1131    1 -970     1        160 -1291 -1402 -1451
#> B    -823   0 -1069    0 -927     0        142 -1211 -1312 -1352
```

3 Positive and negative learning rates

The standard delta rule assumes people adjust Q-values equally when reward is higher than expected compared to when it is lower than expected. However, several research papers suggest that people show an optimism bias: Positive prediction errors lead to larger Q-value adjustments than negative adjustments. This is formalized by allowing for two separate learning rates:

$$PE_t = r_t - Q_t$$
$$Q_{t+1} = Q_t + \begin{cases} \alpha^+ \cdot PE_t & \text{if } PE_t > 0 \\ \alpha^- \cdot PE_t & \text{if } PE_t < 0 \end{cases}$$

In *EMC2*, this is implemented by using the `'delta2lr'` kernel type, which introduces two learning rate parameters (`v.alphaPos` and `v.alphaNeg`) instead of one. We demonstrate this using the openly shared data from Lefebvre et al. (2017).

```r
load('./data/lefebvre2017/lefebvre_exp1.RData')
dat$exposure <- get_exposure(dat)

all_symbols <- unique(c(dat$s_left, dat$s_right))
make_covariate_columns <- setNames(lapply(all_symbols,
                                          covariate_column_generator), all_symbols)

kernel <- make_kernel(all_symbols, 'delta2lr')
base1 <- make_base('v', 'lin', kernel=kernel, phase='posttransform',
coding=list('d'=make_RL_ARD_difference))
base2 <- make_base('v', 'lin', kernel=kernel, phase='posttransform',
coding=list('s'=make_RL_ARD_sum))
trend <- make_trend(base1, base2)

ADmat <- matrix(c(1, -1), ncol=1, dimnames=list(NULL, c('a-s')))
design_RDM <- design(model=RDM,
                     data=dat,
                     functions=make_covariate_columns,
                     contrasts=list(cue=ADmat),
                     formula=list(B ~ 1, v ~ 1, t0 ~ 1),
                     trend=trend,
                     constants=c('v.q0'=0),
                     report_p_vector=TRUE)
```

```
#>
#>  Sampled Parameters:
#> [1] "B"          "v"          "t0"         "v.alphaPos" "v.alphaNeg"
#> [6] "v.w_d"      "v.w_s"
#>
#>  Design Matrices:
#> $B
#>  B
#>  1
#>
#> $v
#>  v
#>  1
#>
#> $t0
#>  t0
#>   1
#>
#> $v.q0
#>  v.q0
#>     1
#>
#> $v.alphaPos
#>  v.alphaPos
#>           1
#>
#> $v.alphaNeg
#>  v.alphaNeg
#>           1
#>
#> $v.w_d
#>  v.w_d
#>      1
#>
```

```
#> $v.w_s
#>  v.w_s
#>      1
#>
#> $A
#>  A
#>  1
#>
#> $s
#>  s
#>  1
```

```
emc <- make_emc(dat, design_RDM)
```

We can inspect the group-level posterior parameters, which indeed suggest that there is a difference between the positive (group-level mean ~ 0.17) and negative (~0.064) learning rates:

```
emc <- fit(emc)
check(emc)
credint(emc)
```

```
#> Iterations:
#>      preburn burn adapt sample
#> [1,]       0    0     0   1000
#> [2,]       0    0     0   1000
#> [3,]       0    0     0   1000
#>
#>  mu
#>            B       v      t0 v.alphaPos v.alphaNeg   v.w_d    v.w_s
#> Rhat   1.009   1.007   1.015      1.017      1.011   1.002    1.002
#> ESS  807.000 716.000 406.000    390.000    588.000 613.000 1494.000


#>
#>  sigma2
#>        B      v      t0 v.alphaPos v.alphaNeg   v.w_d   v.w_s
#> Rhat   1   1.01   1.013      1.011      1.014   1.008   1.005
#> ESS  644 748.00 391.000    402.000    385.000 636.000 948.000


#>
#>  alpha highest Rhat : 30
#>             B        v      t0 v.alphaPos v.alphaNeg    v.w_d v.w_s
#> Rhat    1.018    1.008   1.013      1.011      1.008    1.005     1
#> ESS  1188.000 2117.000 778.000    691.000   1394.000 1064.000  2110
```

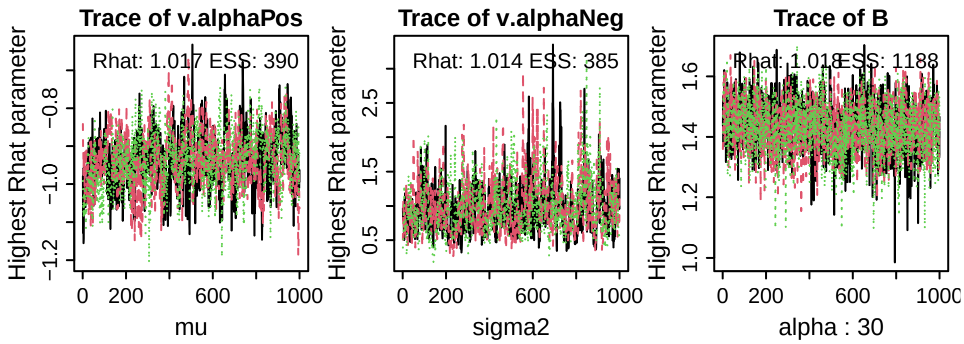


Figure 12: Trace plots of the sampled parameters with highest Rhat values (i.e., worst convergence) at the group-level mean (left, mu), the group-level variance (middle, sigma2), and the subject-level (alpha; here, subject 7). ESS = effective sample size.

```r
credint(emc)
```

```
#> $mu
#>               2.5%    50%  97.5%
#> B            1.020  1.117  1.212
#> v            0.460  0.548  0.640
#> t0          -0.973 -0.819 -0.689
#> v.alphaPos -1.080 -0.936 -0.792
#> v.alphaNeg -1.921 -1.523 -1.168
#> v.w_d        1.420  1.655  1.898
#> v.w_s        0.351  0.564  0.774
```

3.0.1 Reparameterisation: estimating the learning rate difference directly

The two-learning-rate model above is useful for model comparison — for instance, to demonstrate that two learning rates provide a more parsimonious account of the data than one. However, it does not directly estimate the *difference* between learning rates, which is often the target of inference. A better parameterisation expresses the two rates as a mean and a difference:

$$\alpha^{+} = \alpha_{\text{mean}} + \tfrac{1}{2}\,\alpha_{\text{diff}}, \qquad \alpha^{-} = \alpha_{\text{mean}} - \tfrac{1}{2}\,\alpha_{\text{diff}}$$

This allows a prior to be placed directly on

$$\alpha_{\text{diff}}$$

and yields participant-wise estimates of the asymmetry. In *EMC2*, linear reparameterisations of kernel parameters are specified via the `parameter_design` argument to `design()`. The reparameterisation matrix maps the sampled parameters (`alphaMean`, `alphaDiff`) to the kernel parameters (`v.alphaPos`, `v.alphaNeg`):

```r
parameter_design <- matrix(c(1,  0.5,
                             1, -0.5),
                           nrow = 2, byrow = TRUE,
                           dimnames = list(c("v.alphaPos", "v.alphaNeg"),
                                           c("alphaMean",  "alphaDiff")))

design_2lr_reparam <- design(model            = RDM,
                             data             = dat,
                             functions        = make_covariate_columns,
                             formula          = list(B ~ 1, v ~ 1, t0 ~ 1),
                             trend            = trend,
                             constants        = c('v.q0' = 0),
                             parameter_design = list(weights = parameter_design),
                             report_p_vector  = FALSE)
```

```r
emc_2lr_reparam <- make_emc(dat, design_2lr_reparam)
```

```r
emc_2lr_reparam <- fit(emc_2lr_reparam)
```

We can now inspect the posterior over the learning rate difference directly, and extract participant-wise estimates:

```r
plot_pars(emc_2lr_reparam)
credint(emc_2lr_reparam)
head(parameters(emc_2lr_reparam, selection = 'alpha'))
```

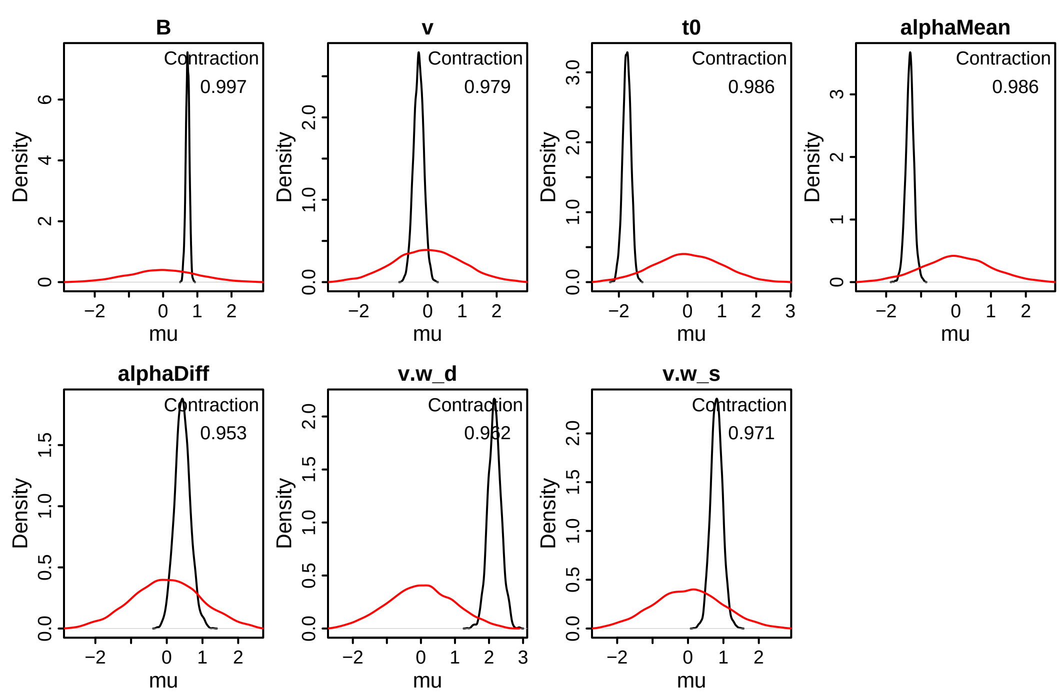


Figure 13: Posterior density (black) and prior density (red) of sampled group-level mean parameters of the reparameterised two-learning-rate RL-ARD. `alphaMean` is the mean learning rate; `alphaDiff` is the difference (positive minus negative learning rate). A positive `alphaDiff` indicates an optimism bias.

```r
credint(emc_2lr_reparam)
```

```
#> $mu
#>              2.5%    50%  97.5%
#> B           0.599  0.715  0.817
#> v          -0.549 -0.263  0.013
#> t0         -2.005 -1.766 -1.542
#> alphaMean -1.557 -1.321 -1.089
#> alphaDiff  0.022  0.438  0.889
#> v.w_d      1.781  2.159  2.561
#> v.w_s      0.484  0.808  1.143
```

```r
head(parameters(emc_2lr_reparam, selection = 'alpha'))
```

```
#>   subjects         B          v        t0 alphaMean    alphaDiff    v.w_d
#> 1       21 0.7501583 -0.4837029 -2.336894 -1.015055 -0.039669604 2.001231
#> 2       21 0.7911816 -0.5550946 -2.143399 -1.014803 -0.052129947 2.132966
#> 3       21 0.7714300 -0.4298633 -2.092540 -1.209469 -0.227846390 2.724979
#> 4       21 0.6375104 -0.2760592 -1.598874 -1.259569  0.003274079 2.571318
#> 5       21 0.5861515 -0.6424500 -1.515646 -1.132896 -0.507824967 2.266387
#> 6       21 0.7523904 -0.3819546 -1.731173 -1.060303 -0.144193501 2.174293
#>       v.w_s
#> 1 0.9808925
#> 2 1.1459935
#> 3 1.3691959
#> 4 1.0058051
#> 5 1.5123174
#> 6 1.3406526
```

4 Factual and counterfactual feedback

An alternative proposal (Palminteri et al. 2017) is that people don't show 'optimism' bias per se, but rather confirmation bias: Any feedback that confirms the current belief processed with a higher learning rate than feedback that opposes current beliefs. If we consider choosing left as expressing a belief that left is more valuable, than this also leads to 'optimism' bias: higher learning rates for better-than-expected rewards (positive RPEs) compared to less-than-expected (negative RPEs). The case of counterfactual feedback can be used to dissociate these two accounts. In an experiment with counterfactual feedback, feedback is presented on both choice options after each choice. The confirmation-bias account suggests that for *chosen* options, the learning rate for positive RPEs should be higher than for negative RPEs. However, for *unchosen* options, the reverse should be the case. That is because not-choosing an option implies believing it is less valuable, and in that case, a less-than-expected RPE confirms that belief, and thus should have a higher learning rate.

As a finale example, we demonstrate how to implement this in *EMC2*. Here, we use the data from the second experiment from Palminteri et al. (2017), which contains factual and counterfactual feedback.

```r
load('./data/palminteri2017_exp2.RData')

all_symbols <- unique(c(dat$s_left, dat$s_right))
all_symbols <- all_symbols[order(as.numeric(gsub('[^0-9]', '', all_symbols)))]

# The feedback now differs between accumulators. For that reason, we need a new covariate
column generator
covariate_column_generator <- function(col_name) {
  f <- function(data) {
    out <- rep(NA, nrow(data))

    is_left  <- data$lR == "left"  & data$s_left  == col_name
    is_right <- data$lR == "right" & data$s_right == col_name

    out[is_left]  <- data$reward_left[is_left]
    out[is_right] <- data$reward_right[is_right]

    out
  }
  return(f)
}

make_covariate_columns <- setNames(lapply(all_symbols,
                                          covariate_column_generator), all_symbols)

# We also need to present feedback to both choice options when simulation new data. Hence, we
build two feedback generators
feedback_generator_left <- function(d) {
  if ("lR" %in% names(d) && nlevels(d$lR) > 1) {
    data        <- d[d$lR == levels(d$lR)[1], ]
    reward_left <- rbinom(nrow(data), 1, data$p_left)
    rep(reward_left, each = nlevels(d$lR))
  } else {
    rbinom(nrow(d), 1, d$p_left)
  }
}

feedback_generator_right <- function(d) {
  if ("lR" %in% names(d) && nlevels(d$lR) > 1) {
    data         <- d[d$lR == levels(d$lR)[1], ]
    reward_right <- rbinom(nrow(data), 1, data$p_right)
    rep(reward_right, each = nlevels(d$lR))
  } else {
    rbinom(nrow(d), 1, d$p_right)
  }
}

kernel <- make_kernel(all_symbols, 'delta2lr', at_mode='push')
base1 <- make_base('v', 'lin', kernel=kernel, phase='posttransform',
coding=list('d'=make_RL_ARD_difference))
base2 <- make_base('v', 'lin', kernel=kernel, phase='posttransform',
coding=list('s'=make_RL_ARD_sum))
trend <- make_trend(base1, base2)

ADmat <- matrix(c(1, -1), ncol=1, dimnames=list(NULL, c('a-s')))
design_RDM <- design(model=RDM,
                     data=dat,
                     functions=c(chosen=function(x) x$R==x$lR,
                                 reward_left=feedback_generator_left,
                                 reward_right=feedback_generator_right,
                                 make_covariate_columns),
                     formula=list(B ~ 1, v ~ 1, t0 ~ 1, v.alphaPos~chosen-1,
                     v.alphaNeg~chosen-1),
                     trend=trend,
                     constants=c('v.q0'=0),
                     report_p_vector=TRUE)
```

```
#>
#>  Sampled Parameters:
#> [1] "B"                        "v"                       "t0"
#> [4] "v.alphaPos_chosenFALSE" "v.alphaPos_chosenTRUE"  "v.alphaNeg_chosenFALSE"
#> [7] "v.alphaNeg_chosenTRUE"  "v.w_d"                  "v.w_s"
#>
#>  Design Matrices:
#> $B
#>  B
#>  1
#>
#> $v
#>  v
#>  1
#>
#> $t0
#>  t0
#>   1
#>
#> $v.alphaPos
#>  chosen v.alphaPos_chosenFALSE v.alphaPos_chosenTRUE
#>    TRUE                      0                     1
#>   FALSE                      1                     0
#>
#> $v.alphaNeg
#>  chosen v.alphaNeg_chosenFALSE v.alphaNeg_chosenTRUE
#>    TRUE                      0                     1
#>   FALSE                      1                     0
#>
#> $v.q0
#>  v.q0
#>     1
#>
#> $v.w_d
#>  v.w_d
#>      1
#>
#> $v.w_s
#>  v.w_s
#>      1
#>
#> $A
#>  A
#>  1
#>
#> $s
#>  s
#>  1
```

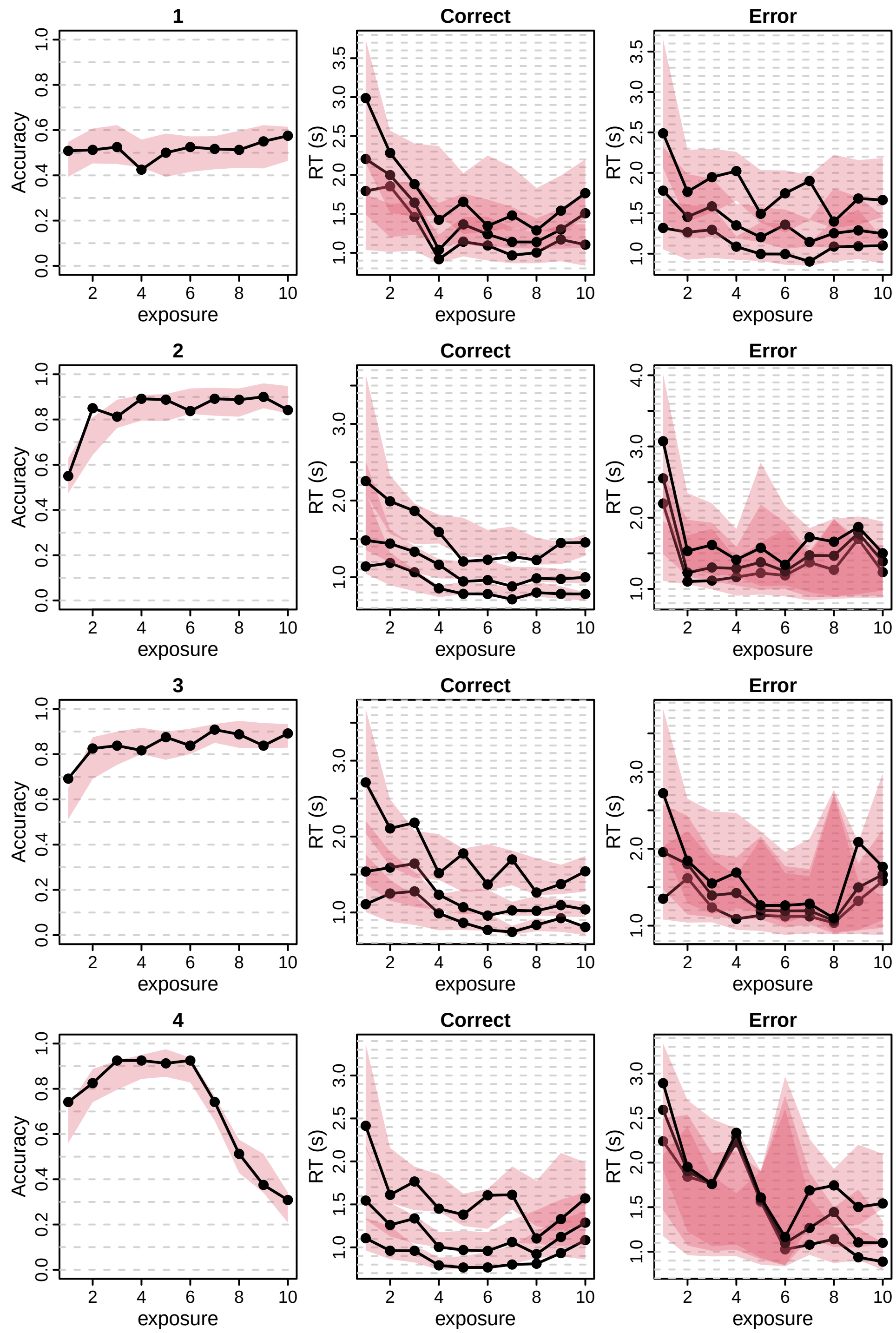
```
emc <- make_emc(dat, design_RDM)
```

We can fit as normal:

```
emc <- fit(emc)
```

And generate posterior predictives and plot. Note that two of the conditions are difficult to plot with accuracy-coding of the choices: The first condition has equal reward probabilities for all symbols, so there is no “correct” answer in any trial. The fourth condition has a reversal, where the reward probabilities switch between symbols after 13 trials. So instead of plotting the accuracy, we can simply plot the probability of choosing ‘right’ in the first column, and plot conditions in rows:

```
pp <- predict(emc)
dat$choice <- dat$R=='right'
pp$choice <- pp$R=='right'
plot_learning(dat, pp[is.finite(pp$rt),], x.var='exposure', acc.var = 'choice',
              acc.ylim = c(0, 1), row.factor = 'condition')
```



And what are the resulting parameters?

```r
credint(emc, map=TRUE)
```

```
#> $mu
#>                             2.5%    50% 97.5%
```

```
#> v                        1.211 1.263 1.317
#> B                        2.016 2.100 2.194
#> t0                       0.167 0.193 0.216
#> v.w_d                    1.296 1.377 1.458
#> v.alphaPos_chosenFALSE 0.058 0.086 0.115
#> v.alphaPos_chosenTRUE  0.401 0.446 0.488
#> v.alphaNeg_chosenFALSE 0.638 0.711 0.779
#> v.alphaNeg_chosenTRUE  0.061 0.081 0.103
#> v.w_s                    0.611 0.706 0.806
```

Which indeed shows the expected interaction between learning rate valence and chosen/unchosen.

Lefebvre, Germain, Maël Lebreton, Florent Meyniel, Sacha Bourgeois-Gironde, and Stefano Palminteri. 2017. "Behavioural and Neural Characterization of Optimistic Reinforcement Learning." *Nature Human Behaviour* 1 (4): 1–9. https://doi.org/10.1038/s41562-017-0067.

Miletić, Steven, Russell J. Boag, Anne C. Trutti, Niek Stevenson, Birte U. Forstmann, and Andrew Heathcote. 2021. "A New Model of Decision Processing in Instrumental Learning Tasks." *eLife* 10 (January): 1–33. https://doi.org/10.7554/eLife.63055.

Miletić, Steven, Niek Stevenson, Luke J G Strickland, and Andrew Heathcote. 2026. "Modelling Structured Trial-by-Trial Variability in Evidence Accumulation." PsyArXiv.

Palminteri, Stefano, Germain Lefebvre, Emma J. Kilford, and Sarah-Jayne Blakemore. 2017. "Confirmation Bias in Human Reinforcement Learning: Evidence from Counterfactual Feedback Processing." *PLOS Computational Biology* 13 (8): e1005684. https://doi.org/10.1371/journal.pcbi.1005684.

Stevenson, Niek, Michelle C. Donzallaz, Reilly J. Innes, Birte U. Forstmann, Dora Matzke, and Andrew Heathcote. 2026. "Bayesian Hierarchical Cognitive Modeling with the EMC2 Package." *Behavior Research Methods* 58 (1): 35. https://doi.org/10.3758/s13428-025-02869-y.